MPJ – Mini Project

VoteChain

Secure Blockchain-Based E-Voting Platform for Maharashtra State Elections

Core Language
Java JDK 17+

Cryptography
SHA-256 + RSA-2048

Consensus
Proof of Work

Project Guide
Prof. Suhas Joshi

Presented by

Name	PRN	Roll No.
Kunal Tailor	1032233258	38
Rishav Singh	1032233012	34
Prakash	1032233203	40
Anuj Kudu	1032233203	36

Date
April 2026

Problem Analysis

Background & Problem Statement

Critical vulnerabilities in centralized database voting systems

Zero Immutability

Centralized databases allow UPDATE and DELETE operations. A malicious DBA or hacker with SQL access can modify vote counts, change candidate selections, or delete records entirely with no audit trail.

DBA Tampering Risk

Database Administrators possess root-level access. Without cryptographic verification, there's no mechanism to detect if a privileged insider modifies vote data for political or financial gain.

No Non-Repudiation

Traditional systems cannot cryptographically prove that a specific voter cast a specific vote. Voters can deny their participation, and the system cannot provide legal-grade evidence of voter intent.

Lack of Transparency

Citizens cannot independently verify election integrity. Vote counting happens behind closed doors, requiring trust in election officials rather than mathematical verification.

The Blockchain Solution

Blockchain replaces the centralized database with an **append-only, cryptographically linked data structure** where each vote block contains the hash of its predecessor. Any tampering breaks the entire chain, providing immediate mathematical detection of fraud.

System Design

Proposed Architecture: The Five-Layer Stack

Clean separation of concerns with modular, testable components

UI Layer

- JavaFX FXML screens
- Scene Builder layouts
- CSS styling

Service Layer

- Business logic
- Block mining
- Validation

Security Layer

- SHA-256 hashing
- RSA-2048 signatures
- Base64 encoding

Storage Layer

- Java Serialization
- blockchain.dat
- voters.dat

Integration Layer

- HttpClient
- JSON-RPC
- GetBlock node

Data Flow Architecture

Voter (JavaFX UI) → **Service** (Business Logic) → **Security** (Hash + Sign) → **Storage** (.dat Files) → **Web3** (GetBlock)

Paradigm Shift

Core Philosophy: Blockchain as Immutable Ledger

Discarding relational databases for cryptographically chained data structures

Traditional Database

- **CRUD Operations:** CREATE, READ, UPDATE, DELETE
- **Mutable Records:** Votes can be modified
- **No Audit Trail:** Changes are invisible
- **Centralized Trust:** DBA is single point of failure

```
SQL: UPDATE votes SET candidate='X' WHERE id=123;
```

Blockchain Ledger

- **Append-Only:** Only CREATE operation allowed
- **Immutable Records:** Votes cannot be altered
- **Cryptographic Audit:** Every change breaks chain
- **Distributed Trust:** Math, not humans

```
blockchain.add(new Block(data, prevHash));
```

The Chain Link Principle

Mathematical Guarantee: If Block N is tampered, its hash changes → Block N+1's prevHash mismatch → Chain break detected instantly

Block Structure

Anatomy of a Vote Block: Block.java Deep Dive

Complete variable structure and hash calculation formula

Block.java Variables

int index	Block position
String voterId	Unique voter ID
String candidateName	Selected candidate
String timestamp	Vote time
String previousHash	Link to prior block
String currentHash	This block's hash
int nonce	PoW iterator
long miningTime	Mining duration
byte[] digitalSignature	RSA signature
String publicKey	Base64 encoded

calculateHash() Method

```
public String calculateHash() { String data = index + voterId + candidateName + timestamp + previousHash + nonce; return applySha256(data); }
```

Concatenation Formula

All fields concatenated as single string before hashing

SHA-256 Output

256-bit hash = 64 hexadecimal characters

Nonce Impact

Changing nonce produces completely different hash

Hash Calculation Example

Input String

1VOTER123CANDIDATE_A2024-04-16T10:30:00000a3f...42

SHA-256 Output

000b7c8d9e2f1a4b5c6d7e8f9a0b1c2d3e4f5a6b7c8d9e0f1a2b3c4d5e6f7a8

Security Layer 1

SHA-256 Cryptographic Hashing Algorithm

The mathematical foundation of blockchain immutability

Why SHA-256?

Avalanche Effect

A **single bit change** in input produces a **completely different** 256-bit output. This ensures any tampering is immediately visible.

Collision Resistance

Computationally infeasible to find two different inputs that produce the same hash output (2^256 possible combinations).

One-Way Function

Impossible to reverse-engineer the original data from the hash. This protects voter privacy while maintaining integrity.

Java Implementation

```
import java.security.MessageDigest; public class HashUtil { public static String applySha256(String input) { try { MessageDigest digest = MessageDigest.getInstance("SHA-256"); byte[] hash = digest.digest(input.getBytes("UTF-8")); StringBuilder hexString = new StringBuilder(); for (byte b : hash) { String hex = Integer.toHexString(0xff & b); if (hex.length() == 1) hexString.append('0'); hexString.append(hex); } return hexString.toString(); } catch (Exception e) { throw new RuntimeException(e); } } }
```

MessageDigest Class
Java Cryptography Architecture (JCA) provider

UTF-8 Encoding
Ensures consistent byte representation across platforms

Hexadecimal Output
64-character hex string = 256 bits of data

Avalanche Effect Demonstration

Original Input
"VOTE:CandidateA"

SHA-256 Output
a3f5c8d2e1b9...

Modified Input (1 char)
"VOTE:CandidateB"

SHA-256 Output
7e9a4b2c8f3d...

Result: Only 1 bit changed in input → 50% of output bits flipped. Tampering is mathematically impossible to hide.

Security Layer 2

Proof of Work (PoW) Mining Algorithm

Computational proof as spam prevention and consensus mechanism

Mining Algorithm

1. **Initialize nonce = 0**
   Start with zero iterator
2. **Calculate hash with current nonce**
   Apply SHA-256 to concatenated data
3. **Check difficulty target**
   Does hash start with N zeros?
4. **If match → Block mined!**
   Save nonce and hash
5. **If no match → nonce++**
   Repeat from step 2

mineBlock() Implementation

```
public void mineBlock(int difficulty) { String target = new String(new char[difficulty]) .replace('\0', '0');
long startTime = System.currentTimeMillis(); while(!currentHash.substring(0, difficulty)
.equals(target)) { nonce++; currentHash = calculateHash(); } miningTime = System.currentTimeMillis()
- startTime; System.out.println("Block Mined: " + currentHash); }
```

Dynamic Difficulty
Target string = "000...0" (N zeros)

while(true) Loop
Infinite iteration until target met

Mining Time Metric
Performance measurement in ms

Difficulty Level	Avg. Iterations	Avg. Mining Time
4	~65,536	~50ms
Target: "0000"	16^4 combinations	On modern CPU

Security Layer 3

RSA-2048 Digital Signatures

Cryptographic non-repudiation using Public Key Infrastructure (PKI)

RSA Key Generation

```
KeyPairGenerator keyGen = KeyPairGenerator.getInstance("RSA"); keyGen.initialize(2048); KeyPair keyPair = keyGen.generateKeyPair(); PrivateKey privateKey = keyPair.getPrivate(); PublicKey publicKey = keyPair.getPublic();
```

2048-bit Key Size

Military-grade encryption strength. Estimated 1 billion years to crack with current technology.

KeyPairGenerator

Java Cryptography Architecture (JCA) standard provider

Signing & Verification

```
// Sign data with private key Signature signer = Signature.getInstance("SHA256withRSA"); signer.initSign(privateKey); signer.update(voteData.getBytes()); byte[] signature = signer.sign(); // Verify with public key Signature verifier = Signature.getInstance("SHA256withRSA"); verifier.initVerify(publicKey); verifier.update(voteData.getBytes()); boolean valid = verifier.verify(signature);
```

SHA256withRSA

Combines SHA-256 hashing with RSA encryption for maximum security

Non-Repudiation

Only the private key holder can create valid signatures

Digital Signature Workflow

Voter → Sign Vote → **Signed Block** → Verify → **Blockchain**

Has Private Key | Vote + Signature | Stores Public Key

Edge Case Management

Genesis Block Integrity & Synchronization

Critical production logic preventing chain-link shattering across serialization reboots

The Problem

Dynamic Timestamp Issue

If Genesis block uses `LocalDateTime.now()`:

- ✗ Every restart = new timestamp
- ✗ New timestamp = different hash
- ✗ Block 1's prevHash ≠ Genesis hash
- ✗ Chain flagged as TAMPERED!

Impact

Fresh chain incorrectly marked as corrupted on every application startup

The Solution

Hardcoded Timestamp

```
public Block() { // Genesis block constructor this.index = 0; this.timestamp = "2024-01-01 00:00:00"; this.previousHash = "0"; this.voterId = "GENESIS"; this.candidateName = "Genesis Block"; this.currentHash = calculateHash(); }
```

Fixed timestamp ensures **identical Genesis hash** across every restart

Benefits

- ✓ Consistent Genesis hash forever
- ✓ Block 1's prevHash always valid
- ✓ Chain integrity preserved

Chain Synchronization Visualization

Storage Layer

State Persistence: Java Object Serialization

Native Java serialization for complete blockchain state persistence without SQL

Serializable Interface

```
public class Block implements Serializable { private static final long serialVersionUID = 1L; // All fields automatically serialized } public class Blockchain implements Serializable { private static final long serialVersionUID = 1L; private List chain; // Entire chain persisted as single object }
```

Serializable Marker

JVM handles object-to-byte conversion automatically

serialVersionUID

Ensures version compatibility during deserialization

Persistence Implementation

```
// Save blockchain to file public void saveBlockchain() { try (ObjectOutputStream oos = new ObjectOutputStream( new FileOutputStream("blockchain.dat"))) { oos.writeObject(blockchain); System.out.println("Blockchain saved!"); } catch (IOException e) { e.printStackTrace(); } } // Load blockchain from file public Blockchain loadBlockchain() { try (ObjectInputStream ois = new ObjectInputStream( new FileInputStream("blockchain.dat"))) { return (Blockchain) ois.readObject(); } catch (Exception e) { return new Blockchain(); // Return new chain } }
```

ObjectOutputStream

Writes complete object graph to binary file

ObjectInputStream

Reconstructs objects with all relationships intact

File Structure

blockchain.dat	Complete chain
voters.dat	Voter registry

Advantages

- **Zero SQL:** No database dependencies
- **Object Integrity:** All relationships preserved
- **Fast I/O:** Binary format = efficient
- **Simple:** Native Java, no external libs

Integration Layer

Real-World Web3 Connectivity: GetBlock JSON-RPC

Live Ethereum node synchronization proving real blockchain infrastructure integration

JSON-RPC Protocol

Request Payload

```
{ "jsonrpc": "2.0", "method": "eth_blockNumber", "params": [], "id": 1 }
```

Response Payload

```
{ "jsonrpc": "2.0", "id": 1, "result": "0x14a2c3d" }
```

GetBlockClient.java

```
import java.net.http.HttpClient; import java.net.http.HttpRequest; import java.net.http.HttpResponse; public class GetBlockClient { private static final String API_URL = "https://go.getblock.io/"; public long getBlockNumber() throws Exception { HttpClient client = HttpClient.newHttpClient(); String jsonPayload = "{" + "\"jsonrpc\":\"2.0\"," + "\"method\":\"eth_blockNumber\"," + "\"params\":[]," + "\"id\":1}"; HttpRequest request = HttpRequest.newBuilder() .uri(URI.create(API_URL)) .header("Content-Type", "application/json") .POST(HttpRequest.BodyPublishers.ofString(jsonPayload)) .build(); HttpResponse response = client.send(request, HttpResponse.BodyHandlers.ofString()); // Parse JSON and extract hex result String hexBlockNum = extractResult(response.body()); return Long.parseLong(hexBlockNum.substring(2), 16); } }
```

HttpClient (Java 11+)

Modern async HTTP client, replaces Apache HttpClient

Hex to Long Conversion

Long.parseLong(hex, 16) converts 0x14a2c3d to decimal

Web3 Integration Flow

VoteChain App
JavaFX Client

→ HTTP POST

GetBlock API
JSON-RPC Gateway

→ Relay

Ethereum Node
Live Network

Application Flow

Voter Subsystem: JavaFX UI Trajectory

End-to-end voter journey from authentication to signed confirmation

1 Authentication

- Voter enters ID & password
- System validates credentials
- HashMap O(1) lookup

2 Ballot Selection

- Active elections displayed
- Candidate list from HashMap
- Party symbols shown

3 Vote Casting

- Voter selects candidate
- Duplicate check (HashMap)
- Block creation triggered

4 RSA Signing

- DigitalSignatureUtil called
- Vote signed with private key
- SHA256withRSA algorithm

5 Mining

- PoW mining executed
- Nonce incremented
- Target difficulty met

Duplicate Prevention Logic

```
// HashMap for O(1) duplicate detection private Map votedVoters = new HashMap<>(); public boolean castVote(String voterId, String candidate) { // Check if already voted if (votedVoters.containsKey(voterId)) { return false; // Duplicate vote rejected } // Create and mine block Block newBlock = new Block(...); newBlock.mineBlock(difficulty); // Mark as voted votedVoters.put(voterId, true); return true; }
```

Confirmation Screen

Vote Successfully Recorded!

Block Index: 42

Block Hash: 000b7c...

Mining Time: 47ms

Signature: Verified ✓

Your vote is permanent and cryptographically secured. The block has been added to the blockchain and cannot be altered.

Application Flow

Admin Subsystem: Command Center

Administrative tools for monitoring, validation, and Web3 testing

Live Analytics

- ✓ Real-time vote totals
- ✓ Party-wise pie charts
- ✓ Turnout percentages
- ✓ JavaFX Charts API

Blockchain Explorer

- ✓ Full chain traversal
- ✓ Hash-linked visualization
- ✓ Block details view
- ✓ Genesis to latest block

Validation

- ✓ validateBlockchain() call
- ✓ Hash verification
- ✓ Chain link validation
- ✓ Tamper detection

validateBlockchain() Implementation

```
public boolean validateBlockchain() { for (int i = 1; i < chain.size(); i++) { Block current = chain.get(i); Block previous = chain.get(i - 1); // Check 1: Current hash validity if (!current.getCurrentHash() .equals(current.calculateHash())) { System.out.println("TAMPERED: Invalid hash at block " + i); return false; } // Check 2: Chain link validity if (!current.getPreviousHash() .equals(previous.getCurrentHash())) { System.out.println("TAMPERED: Chain broken at block " + i); return false; } } return true; }
```

Web3 Test Function

GetBlock Connectivity Test

Admin can verify live Ethereum node connection:

```
GetBlockClient client = new GetBlockClient(); long blockNum = client.getBlockNumber(); System.out.println("Live Ethereum Block: " + blockNum);
```

Output: "Live Ethereum Block: 21523453"

Confirms active Web3 connectivity

Quality Assurance

System Tolerances & Edge Case Testing

Robustness validation through attack simulation and boundary testing

Tamper Detection Test

Attack Simulation

1. Stop VoteChain application
2. Open blockchain.dat in hex editor
3. Modify Block 5's candidate field
4. Save and restart application

Expected Result

```
validateBlockchain() called... TAMPERED: Invalid hash at block 5 Chain validation: FAILED
```

Why it works: Modified data → Different hash → Hash mismatch detected → Tamper flag raised

Duplicate Vote Prevention Test

Test Scenario

1. Voter "V123" casts vote for Candidate A
2. Block mined and added to chain
3. Voter "V123" attempts second vote
4. System checks HashMap containsKey()

Expected Result

```
if (votedVoters.containsKey("V123")) { return false; // Rejected! } // Output: "Error: Already voted"
```

Performance: HashMap O(1) lookup = instant duplicate detection

Genesis Integrity Test

- ✓ Verify hardcoded timestamp
- ✓ Consistent hash across restarts
- ✓ Block 1 prevHash valid

Mining Timeout Test

- ✓ Extreme difficulty levels
- ✓ Nonce overflow protection
- ✓ Performance benchmarks

Web3 Connectivity Test

- ✓ GetBlock API reachable
- ✓ JSON-RPC response valid
- ✓ Hex to decimal conversion

Project Summary

Conclusion, Limitations & Future Enhancements

Project Achievements

- **Complete Java Blockchain**
  Full implementation from scratch with JDK 17+
- **SHA-256 + RSA-2048 Security**
  Military-grade cryptography implementation
- **Proof of Work Mining**
  Dynamic difficulty with nonce iteration
- **Native Java Serialization**
  Zero SQL dependency, pure object persistence
- **Live Web3 Connectivity**
  GetBlock JSON-RPC integration
- **JavaFX Desktop UI**
  Professional voter and admin interfaces

Current Limitations

- **Single-Node Architecture**
  Not distributed - single point of failure
- **Local File Storage**
  No cloud backup or redundancy
- **No Smart Contracts**
  Business logic in Java, not Solidity

Future Roadmap

- **P2P Network Gossiping**
  Distributed consensus across nodes
- **MongoDB Atlas Integration**
  Cloud persistence with redundancy
- **Smart Contract Deployment**
  Solidity contracts on Ethereum mainnet

Presented by

Kunal Tailor

Project

VoteChain